Applied Soft Computing Journal

A Hybrid DeBERTa–RNN–LSTM Model Optimized Using Artificial Bee Colony for Code-Mixed Hate-Speech Detection

--Manuscript Draft--

Manuscript Number:	ASOC-D-26-02463
Article Type:	VSI:ASOC_ABC_20
Keywords:	Artificial Bee Colony; DeBERTa; Hate-Speech Detection; Natural Language Processing; Swarm Intelligence
Corresponding Author:	Sauvik Bal Techno India University
First Author:	Akash Jana
Order of Authors:	Akash Jana
	Shillpi Mishrra
	Sauvik Bal
Abstract:	In linguistically diverse societies like India, where code-mixed language varieties like Hindi-English (Hinglish) are prevalent, the study attempts to deal the problem of hate speech identification on social media platforms. The monolingual categorization process appears to be challenged by the unstructured nature, transliteration, and colloquial words found in code-mixed texts. A hybrid deep learning model that incorporates a combination of sequential feature learning and contextual language modeling is proposed to address these issues. The model uses a combination of LSTM layers and RNNs to recognize patterns in user-generated information and a pre-trained transformer model called DeBERTa for contextual language modeling. The effectiveness of the model is improved by employing the artificial Bee Colony optimization algorithm to explore and optimize the settings of the model, based on the optimization of key parameters. To minimize the level of noise that is inherent in social media writings, the study employs basic preprocessing techniques on real-world Hinglish hate speech datasets. The macro F1 score for the handling of unbalanced datasets defines that the ABC optimized hybrid model give better performance than the traditional deep learning approach. In an effort to enhance the automated content moderation system, the study highlights the effective integration of transformer embeddings, recurrent patterns, and swarm intelligence for hate speech detection in a multilingual environment. With a macro F1 score of 0.6733, test accuracy of 71.73%, test AUC of 0.7951, and best validation AUC of 0.8016, the model shows excellent convergence and generalization, thereby highlighting its effectiveness.

Powered by Editorial Manager® and ProduXion Manager® from Aries Systems Corporation

To
The Editor(s)
20th Anniversary Special Issue of Artificial Bee Colony (ABC) Algorithm: Background, Advances and Future Directions
Applied Soft Computing

Subject: Submission of Manuscript entitled “A Hybrid DeBERTa–RNN–LSTM Model Optimized Using Artificial Bee Colony for Code-Mixed Hate-Speech Detection”

Date: 20.02.2026

Respected Editor(s),

I am pleased to submit our manuscript entitled **“**A Hybrid DeBERTa–RNN–LSTM Model Optimized Using Artificial Bee Colony for Code-Mixed Hate-Speech Detection**”** for consideration in the *Applied Soft Computing* special issue titled **“20th Anniversary Special Issue of Artificial Bee Colony (ABC) Algorithm: Background, Advances and Future Directions.”**

This study addresses hate-speech detection in multilingual, code-mixed contexts, specifically focusing on Hindi–English (Hinglish) text. Recognizing the complexities introduced by informal grammar and contextual ambiguity, the authors propose a hybrid deep learning architecture that utilizes a pretrained DeBERTa transformer alongside RNN and LSTM layers. A significant contribution includes the use of the Artificial Bee Colony (ABC) algorithm for automatic hyperparameter optimization, enhancing model performance without manual tuning. Evaluating the model on real-world datasets, results indicate a test accuracy of 71.73%, a macro F1-score of 0.6733, and a test AUC of 0.7951. Further validation reached an AUC of 0.8016 with moderate computational resources. The study showcases practical applications of ABC in optimizing deep learning approaches for nuanced natural language processing tasks, emphasizing its significance in real-world, high-dimensional scenarios. The manuscript is original and has not been submitted elsewhere, with all authors agreeing to its submission.

We believe that the proposed work will be of interest to the readers of Applied Soft Computing, particularly those working in swarm intelligence, metaheuristic optimization, and intelligent applications of soft computing in natural language processing.

Thank you for your time and consideration. We look forward to your response.

Yours faithfully,

Akash Jana (akashjana7437@gmail.com)

Sauvik Bal (sauvikbal@gmail.com)

Shillpi Mishrra (shllpimishrra91@gmail.com)

Highlights

A Hybrid DeBERTa–RNN–LSTM Model Optimized Using Artificial Bee Colony for Code-Mixed Hate-Speech Detection

Akash Jana, Shillpi Mishrra, Sauvik Bal

- A hybrid DeBERTa-RNN-LSTM framework for Hinglish hate-speech detection.
- Artificial Bee Colony (ABC) used for automated hyperparameter optimization.
- Effectively captures contextual and sequential features in code-mixed text.
- Achieved 71.73% accuracy and validation AUC of 0.8016 on real datasets.
- Demonstrates improved performance for multilingual social media moderation.

Manuscript Click here to access/download;Manuscript;Manuscript.pdf

Click here to view linked References

A Hybrid DeBERTa–RNN–LSTM Model Optimized Using Artificial Bee Colony for Code-Mixed Hate-Speech Detection⋆

Akash Jana[a,1], Shillpi Mishrra[a,2] and Sauvik Bal[a,*,3]

[a]Department of Computer Science and Engineering, Techno India University, West Bengal, , Kolkata, 700091, West Bengal, India

ARTICLE INFO

Keywords:
Artificial Bee Colony
Code-Mixed Text
DeBERTa
Hate-Speech Detection
Hybrid Deep Learning
Hinglish
Natural Language Processing
Swarm Intelligence

ABSTRACT

In linguistically diverse societies like India, where code-mixed language varieties like Hindi-English (Hinglish) are prevalent, the study attempts to deal the problem of hate speech identification on social media platforms. The monolingual categorization process appears to be challenged by the unstructured nature, transliteration, and colloquial words found in code-mixed texts. A hybrid deep learning model that incorporates a combination of sequential feature learning and contextual language modeling is proposed to address these issues. The model uses a combination of LSTM layers and RNNs to recognize patterns in user-generated information and a pre-trained transformer model called DeBERTa for contextual language modeling. The effectiveness of the model is improved by employing the artificial Bee Colony optimization algorithm to explore and optimize the settings of the model, based on the optimization of key parameters. To minimize the level of noise that is inherent in social media writings, the study employs basic preprocessing techniques on real-world Hinglish hate speech datasets. The macro F1 score for the handling of unbalanced datasets defines that the ABC optimized hybrid model give better performance than the traditional deep learning approach. In an effort to enhance the automated content moderation system, the study highlights the effective integration of transformer embeddings, recurrent patterns, and swarm intelligence for hate speech detection in a multilingual environment. With a macro F1 score of 0.6733, test accuracy of 71.73%, test AUC of 0.7951, and best validation AUC of 0.8016, the model shows excellent convergence and generalization, thereby highlighting its effectiveness.

1. Introduction

The exponential growth of social media has brought about a paradigm shift in public discourse and communication, as it enables free dialogue, but simultaneously, it also enables the spread of hate speech and other unpleasant information. This has significant implications for both individual well-being and social cohesion. In the areas of social computing and natural language processing, hate speech detection algorithms are critical. Although many existing hate speech detection algorithms are effective for resource-rich languages such as English, they are not very effective for multilingual settings where language diversity increases the complexity of processing known as Hinglish which is Hindi-English code-mixing. To effectively represent the semantic relationships that exist in code-mixed communications, there is a need for advanced contextual representation methods to handle issues that are unique to code-mixed text, such as noise, colloquial expressions, and cultural nuances.

With the effective learning of contextual representations of words, recent advances in deep learning, particularly in the form of pretrained transformer-based language models such as BERT and DeBERTa, have shown improvements in several NLP tasks. The presence of sequential dependencies, which are relevant in a conversational setting, may have constraints, however. Recurrent neural networks (RNNs) and transformers can be combined to leverage the benefits of both architectures for classification tasks. Hybrid models involve a large number of hyperparameters to be tuned, making it inefficient to tune them manually. Metaheuristic optimization algorithms, specifically the Artificial Bee Colony algorithm, with their ability to balance exploration and exploitation in high-dimensional search spaces, provide an efficient way to automatically search for hyperparameters and maximize model performance Kiran and Findik (2015); Kiran (2021).

The purpose of this research project is to create a new hybrid deep learning model to identify instances of hate speech in code-mixed multilingual settings, with an emphasis on Hinglish data sets. Member methodologies include: an Artificial Bee Colony optimized autonomous hyperparameter tuning, recurrent neural networks, Long Short-Term Memory (LSTM) layers and contextual embeddings based on DeBERTa. To provide relevant and replicable results for this research, the test datasets utilized were from real-world events as no artificial augmentation was applied to the datasets prior to testing. The hybrid architecture and transformer vs. recurrent models, application of swarm based optimization modelling to increase performance in classified results and use of common metrics to test multilingual data are all major contributions to the literature. The document is structured to provide an overview of the literature, a literature review, a background introduction, a discussion regarding the proposed model(s), a description of the experimental environment, results and discussion, and future directions for research on this topic.

⋆

*Corresponding author
akashjana7437@gmail.com (A. Jana); shllpimishrra91@gmail.com (S. Mishrra); sauvikbal@gmail.com (S. Bal)
ORCID(s):
1

2. Related Work

Due to the rise in unpleasant information on social media platforms, the problem of hate speech detection has gained much attention. This paper covers traditional machine learning, deep learning, multilingual processing, and model optimization. The importance of this study is to identify the gaps in the research and the major developments in the field.

2.1. Hate-Speech Detection in Code-Mixed and Multilingual Text

The detection of hate speech has been primarily carried out in monolingual English texts because of the availability of large datasets and a standardized structure of language Davidson, Warmsley, Macy and Weber (2017); Fortuna and Nunes (2018). The initial approaches used lexical, syntactic, and statistical properties, which resulted in reasonable performance in a controlled environment, but these methods fail in the presence of the informal and diverse language used in social interactions Schmidt and Wiegand (2017). With the increasing number of multilingual communities, especially in areas such as India, where code-mixing like Hinglish is common, traditional natural language processing techniques face substantial difficulties Patwa et al. (2021). Code-mixing leads to ungrammatical structures, phonetic spellings, transliteration, and the use of regional slang. The noisy nature of user-generated content on social media, with its use of emojis, hashtags, informal language, and non-standard conventions, further adds to the complexity of hate speech detection Waseem and Hovy (2016). Techniques involving deep learning using recurrent neural networks and transformers for contextual and semantic representation have also gained popularity in recent years as a tool for detecting the implicit offensiveness of hate speech Caselli, Basile, Mitrović and Granitzer (2020); Badjatiya, Gupta, Gupta and Varma (2017). Nonetheless, owing to the presence of non-standard linguistic patterns, these techniques face challenges in code-mixed settings. Their ability to generalize is adversely affected by biases in the dataset Patwa et al. (2021).

Moreover, the perception of hate speech may change depending on the context of the conversation, and thus sentence-level classification is not sufficient Vidgen, Yasseri and Margetts (2019). These challenges highlight the need for improved representation learning methods that can handle the contextual semantics and dependencies in the noisy mixed-language context. Even though there has been significant progress in the area, the problem of hate speech detection in code-mixed and multilingual scenarios remains an open problem.

2.2. Deep Learning and Transformer-Based Approaches

Since deep learning is so effective at learning hierarchical feature representations directly from unstructured text, it has become the most widely used approach for text classification Young, Hazarika, Poria and Cambria (2018). By maintaining contextual information over time, deep neural networks, specifically recurrent neural networks (RNNs) and long short-term memory (LSTM) networks, successfully model sequential data, as opposed to traditional approaches that require the design of hand-crafted features. Nevertheless, challenges such as vanishing gradients make it challenging for RNNs to model long-term dependencies. The performance of natural language processing has been significantly enhanced by attention mechanisms and transformer models, which are capable of parallel processing and modeling direct token relationships.

For applications such as sentiment analysis and hate speech detection, pretrained models such as BERT are very effective at learning contextual embeddings Devlin, Chang, Lee and Toutanova (2019). The interest in hybrid models that combine transformers with RNNs has increased because, although they are very effective, transformers may struggle to adapt to the unstructured patterns that exist in social media Zhang, Robinson and Tepper (2018). While this hybrid model maintains temporal sequences and improves semantic understanding, it also adds complexity and requires extensive hyperparameter tuning, which highlights the need for automated hyperparameter optimization.

2.3. Artificial Bee Colony for Feature Selection and Optimization

In machine learning and pattern recognition, swarm intelligence methods, namely the Artificial Bee Colony algorithm, are often utilized to tackle complex optimization tasks Patel, Tiwari and Patel (2016); Karaboga and Akay (2011). In this approach, the ABC algorithm mimics the foraging behavior of honey bee colonies by leveraging worker, observer, and scout bees to explore and refine potential solutions based on their fitness levels as defined by a fitness function. By achieving a delicate balance between exploration and exploitation, this approach prevents premature convergence and allows for the search of the entire solution space. In high-dimensional data sets, the ABC algorithm has been successfully applied for feature selection, and it has been found to improve the classification accuracy and reduce the complexity of computing Taghizadeh-Mehrjardi, Neupane, Sood and Kumar (2017); Kiran, Hakli, Gunduz and Uguz (2015); Hakli and Kiran (2020).

The adaptive and hybrid approaches of the ABC algorithm enhance exploration and are suitable for large-scale optimization problems, whereas current advancements focus on optimizing search efficiency and convergence properties. The ABC algorithm is applicable to discrete optimization problems in machine learning and natural language processing tasks based on domain-specific modifications, whereas multi-objective variants aim to maximize conflicting objectives such as accuracy and computational complexity. Furthermore, the ABC algorithm has demonstrated potential in deep learning hyperparameter optimization by exploring combinations of parameters while reducing computational complexity compared to more traditional approaches such as grid or random search Kaya, Gorkemli, Akay and Karaboga (2022); Karaman, Karaboga, Pacal, Akay, Basturk, Nalbantoglu, Coskun and Sahin (2023a).

2.4. Metaheuristic Optimization for Deep Learning Models

The performance, convergence, and generalization of deep learning models are highly affected by the choice of hyperparameters such as learning rate, depth, and batch size. In complex models, traditional methods of hyperparameter optimization, such as grid search and trial and error, are often inefficient. Metaheuristic optimization methods are employed for automatic hyperparameter optimization to improve the process by efficiently exploring large search spaces with a limited number of evaluations. Swarm intelligence methods are particularly effective because they mimic natural behaviors to optimize model performance based on validation metrics such as accuracy and macro F1-scoreDedeturk and Akay (2025).

The literature on metaheuristic optimization for neural networks has been dominated by studies on traditional classification problems and simple models, while hybrid models of deep learning, particularly those that involve recurrent networks with transformer encoders, have remained relatively uncharted. There has been far less work on the use of swarm intelligence optimization methods, such as Artificial Bee Colony optimization, for difficult natural language processing problems with noisy and code-mixed social media dataKaraman, Pacal, Basturk, Akay, Nalbantoglu, Coskun, Sahin and Karaboga (2023b). This is particularly true for transformer-recurrent models of code-mixed hate speech detection, where the complexity of the language involved and the transformer models' susceptibility to hyperparameters make efficient automated optimization methods highly desirable.

2.5. Comparative Analysis of Existing Studies

Table 1 summarizes representative studies in terms of language setting, model type, optimization strategy, and limitations.

2.6. Research Gap

From the above discussion, it is evident that existing research either focuses on (i) deep learning architectures without optimization, or (ii) ABC-based optimization for traditional machine learning tasks. Limited work has explored the integration of advanced transformer models, recurrent sequence learning, and swarm-based optimization within a unified framework for code-mixed hate-speech detection. Furthermore, most studies do not address automated hyperparameter tuning for hybrid deep architectures.

To address these limitations, the present work proposes a hybrid DeBERTa–RNN–LSTM model optimized using the Artificial Bee Colony algorithm. The proposed framework aims to improve detection performance by combining contextual representation learning, sequential modeling, and automated hyperparameter optimization for real-world Hinglish social media data.

3. Proposed Methodology

This section presents the proposed hybrid deep learning framework for code-mixed hate-speech detection. The architecture integrates contextual representation learning using a pretrained transformer with sequential modeling through recurrent networks. An Artificial Bee Colony optimization strategy is employed to automatically determine optimal hyperparameter configurations. The proposed framework consists of four tightly integrated components: (i) a preprocessing module for normalization and tokenization of code-mixed text; (ii) a contextual representation module that applies a DeBERTa model to derive contextual and semantic information; (iii) a hybrid learning module that applies RNN and LSTM layers to model temporal dependencies; and (iv) a hyperparameter optimization module that applies an Artificial Bee Colony algorithm to optimize the performance of the model. The preprocessing stage includes removal of URLs, user mentions, emojis, special characters, and normalization of whitespace, followed by lowercase conversion and tokenization using the pretrained DeBERTa tokenizer.

3.1. System Architecture

Figure 1 illustrates the overall architecture of the proposed system. Given an input text sequence, the preprocessing module performs normalization, tokenization, and noise removal. The processed text is then passed to a pretrained DeBERTa model to obtain contextual embeddings. These embeddings are further processed by stacked recurrent layers consisting of a vanilla recurrent neural network followed by a long short-term memory network. Finally, a fully connected classification layer with a softmax activation produces the hate-speech prediction. During training, the parameters of the pretrained DeBERTa model were fine-tuned jointly with the recurrent layers to enable task-specific adaptation to the code-mixed hate-speech domain.

An Artificial Bee Colony methodology for hyperparameter optimization of a hybrid DeBERTa–RNN–LSTM model is described in the Figure 2. It describes the procedure, which begins with initializing a population of candidate solutions, each of which stands for a configuration of hyperparameters. The validation macro F1-score is used as a fitness statistic to evaluate the model's performance. Three stages make up the optimization process: scout bees swap out underperforming configurations for new ones to maintain diversity, spectator bees choose potential solutions based on fitness, and employed bees conduct local searches for better solutions. Each cycle is followed by a termination check to see whether the maximum number of optimization cycles has been reached. If so, the optimal hyperparameter set for the final model training is chosen, improving generalization performance.

Table 1
Summary of Representative Studies in Hate-Speech Detection and ABC-Based Optimization

Study	Language	Model Type	Optimization	Limitations
Bohra et al. (2018) Bohra, Vijay, Singh, Akhtar and Shrivastava (2018)	Hinglish	Dataset creation	None	No modeling framework
Kamble and Joshi (2018) Kamble and Joshi (2018)	Hinglish	Deep neural network	None	Limited contextual modeling
Santosh and Aravind (2019) Santosh and Aravind (2019)	Hinglish	ML classifiers	None	Relies on shallow features
Chopra et al. (2020) Chopra, Sawhney, Mathur and Shah (2020)	Hinglish	ML & profiling	None	Limited deep contextual learning
Jagdale et al. (2024) Jagdale, Inamdar, Takalikar, Khade and Joshi (2024)	Code-mixed	Embedding-based DL	None	No optimization strategy
Uzer et al. (2013) Uzer, Yilmaz and Inan (2013)	Structured data	ML & ABC	ABC (feature selection)	Not applied to NLP
Wang et al. (2020) Wang, Zhang, Sun, Wang and Du (2020)	High-dimensional data	ML & ABC	Multi-objective ABC	Not deep learning
Lin et al. (2021) Lin, Wang, Li, Zhang and Huang (2021)	Classification tasks	Improved ABC	Adaptive ABC	Domain-specific evaluation
Zhang et al. (2024) Zhang, Fu, Gao, Pan and Huang (2024)	Optimization problems	ABC framework	Multi-objective ABC	Not NLP focused
Sharma and Bhalla (2025) Sharma and Bhalla (2025)	Hinglish	Deep learning	None	No metaheuristic optimization

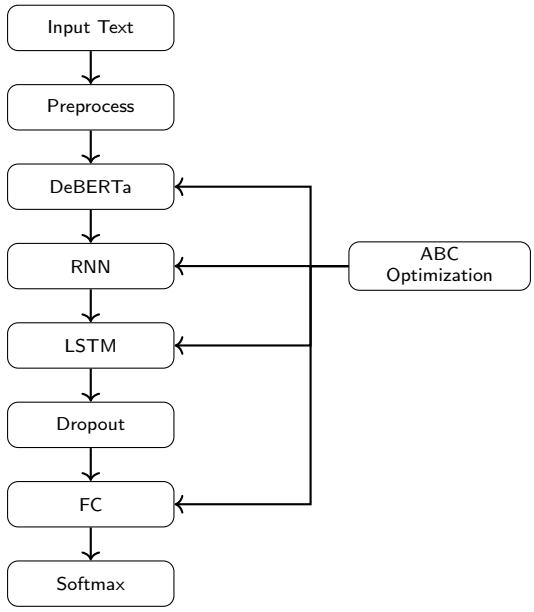

Figure 1: Architecture of the proposed DeBERTa–RNN–LSTM model with ABC-based hyperparameter optimization.

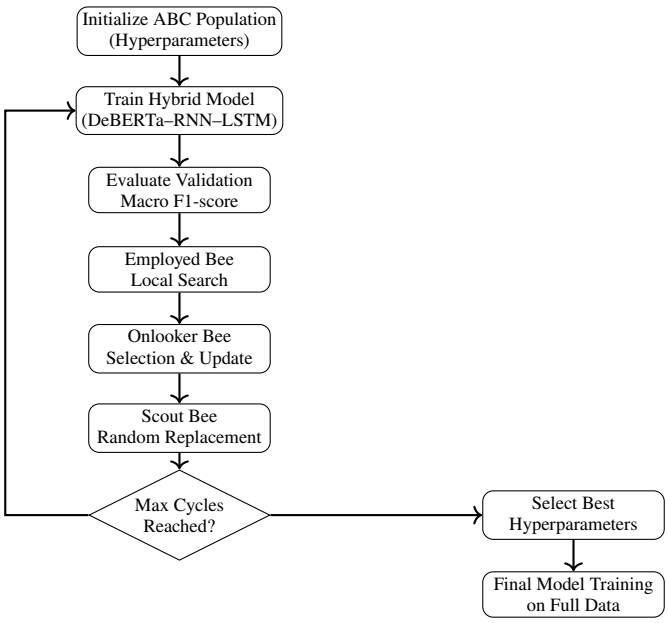

Figure 2: Artificial Bee Colony based hyperparameter optimization process for the proposed hybrid DeBERTa-RNN-LSTM model.

Let a code-mixed input sequence be represented as:

$$X = \{x_1, x_2, \dots, x_T\} \tag{1}$$

where T denotes the sequence length.

The DeBERTa encoder generates contextual representations:

$$H^{(0)} = \text{DeBERTa}(X) \in \mathbb{R}^{T \times d} \tag{2}$$

where d is the embedding dimension. The output of the DeBERTa encoder is fed into the RNN layer with hidden dimension h_r, followed by an LSTM layer with hidden dimension h_l.

The architecture follows a sequential pipeline where contextual embeddings obtained from the pretrained DeBERTa model are further processed through recurrent layers consisting of a vanilla RNN followed by an LSTM layer, and the final hidden representation is passed to a fully connected classification layer with softmax activation.

A dropout layer with rate d_r is applied to the final hidden representation to reduce overfitting before the fully connected classification layer.

3.2. Recurrent Sequential Modeling

The contextual embeddings are passed to a recurrent neural network layer to capture temporal dependencies:

$$H_t^{(1)} = \tanh(W_r H_t^{(0)} + U_r H_{t-1}^{(1)} + b_r) \tag{3}$$

To further model long-range dependencies and mitigate vanishing gradient issues, an LSTM layer is applied:

$$i_t = \sigma(W_i H_t^{(1)} + U_i h_{t-1} + b_i) \tag{4}$$

$$f_t = \sigma(W_f H_t^{(1)} + U_f h_{t-1} + b_f) \tag{5}$$

$$o_t = \sigma(W_o H_t^{(1)} + U_o h_{t-1} + b_o) \tag{6}$$

$$\tilde{c}_t = \tanh(W_c H_t^{(1)} + U_c h_{t-1} + b_c) \tag{7}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{8}$$

$$h_t = o_t \odot \tanh(c_t) \tag{9}$$

The final hidden representation is passed to a dense layer:

$$\hat{y} = \text{Softmax}(W_o h_T + b_o) \tag{10}$$

The model is trained using cross-entropy loss:

$$\mathcal{L} = -\sum_{i=1}^{N} y_i \log(\hat{y}_i) \tag{11}$$

During the optimization process, each candidate configuration is trained for a fixed number of epochs. Model training is performed using standard single-precision floating-point operations, and the validation performance obtained after training is used for evaluating the candidate solution.

3.3. Hyperparameter Optimization Using Artificial Bee Colony

The performance of deep learning models depends significantly on hyperparameter selection. Instead of manual tuning, the proposed framework employs the Artificial Bee Colony algorithm to automatically search for optimal hyperparameter configurations.

The hyperparameter configuration optimized using the Artificial Bee Colony algorithm is represented as a parameter vector, as defined in Eq. (12).

$$\theta = \{\eta, h_r, h_l, b, d_r, e\} \tag{12}$$

In this formulation, η represents the learning rate, h_r and h_l denote the hidden dimensions of the RNN and LSTM layers, respectively, b is the batch size, d_r corresponds to the dropout rate, and e denotes the number of training epochs. The search ranges for the hyperparameters were defined as follows: learning rate $\eta \in [10^{-5}, 10^{-3}]$, RNN hidden size $h_r \in [64, 256]$, LSTM hidden size $h_l \in [64, 256]$, batch size $b \in \{16, 32, 64\}$, dropout rate $d_r \in [0.1, 0.5]$, and number of epochs $e \in [3, 10]$.

Each food source in the ABC algorithm represents a candidate hyperparameter set.

3.3.1. Initialization

An initial population of SN food sources is randomly generated:

$$\theta_i = \theta_{\min} + rand(0,1) \times (\theta_{\max} - \theta_{\min}) \tag{13}$$

3.3.2. Fitness Evaluation

For each candidate configuration, the model is trained and evaluated on validation data. The fitness function is defined as:

$$f(\theta_i) = F1_{macro}(\theta_i) \tag{14}$$

Since the objective is maximization, higher F1 values indicate better food sources. To control computational cost, each candidate configuration was trained for a limited number of epochs during the optimization phase, and the validation macro F1-score obtained at the end of training was used as the fitness value.

3.3.3. *Employed Bee Phase*

Each employed bee generates a neighboring solution:

$$v_{ij} = \theta_{ij} + \phi_{ij}(\theta_{ij} - \theta_{kj}) \tag{15}$$

where $k \neq i$ and $\phi_{ij} \in [-1, 1]$.

If $f(v_i) > f(\theta_i)$, the solution is updated.

3.3.4. *Onlooker Bee Phase*

Onlooker bees select food sources based on probability:

$$p_i = \frac{f(\theta_i)}{\sum_{n=1}^{SN} f(\theta_n)} \tag{16}$$

Selected solutions are further exploited using neighborhood search.

3.3.5. *Scout Bee Phase*

If a solution does not improve for a predefined limit, it is abandoned and replaced with a new random solution:

$$\theta_i = \theta_{\min} + rand(0, 1)(\theta_{\max} - \theta_{\min}) \tag{17}$$

3.4. Overall Optimization Objective

The ABC optimization aims to solve:

$$\theta^* = \arg\max_{\theta} \; F1_{macro}(\theta) \tag{18}$$

The optimal configuration θ^* is then used to train the final hybrid model on the full training dataset.

For each food source representing a hyperparameter configuration, the hybrid model is trained on the training dataset and evaluated on the validation dataset. The macro F1-score obtained on the validation set is used as the fitness value. The objective of the optimization process is to maximize the validation macro F1-score.

3.5. Algorithm Workflow

The overall procedure is summarized in Algorithm 1.

The ABC optimization process was conducted using a population size of $SN = 10$, a maximum of $C_{max} = 20$ optimization cycles, and an abandonment limit of $L = 5$. The computational complexity of the optimization process is proportional to $O(SN \times C_{max} \times T_{train})$, where SN is the population size, C_{max} is the number of optimization cycles, and T_{train} denotes the training time of the hybrid model for a single configuration.

After the optimization phase, the model is retrained using the optimal hyperparameter configuration θ^* on the combined training and validation data.

Algorithm 1 ABC-Based Hyperparameter Optimization for Hybrid DeBERTa-RNN-LSTM

Require: Training data D_{train}, validation data D_{val}, population size SN, cycles C_{max}, limit L
Ensure: Optimal hyperparameters θ^*

```
Define parameter vector: θ = {η, h_r, h_l, b, d_r, e}
Initialize SN solutions randomly within bounds
Set trial_i = 0 for all solutions
for each solution θ_i do
    Train hybrid model on D_train
    Compute fitness f_i using validation macro-F1
end for
for cycle = 1 to C_max do
    Employed Bees
    for i = 1 to SN do
        Select k ≠ i
        Generate neighbor: v_ij = θ_ij + φ(θ_ij − θ_kj)
        Clip bounds and round integer parameters
        Train model with v_i and fitness evaluation
        if f(v_i) > f_i then
            θ_i ← v_i, trial_i = 0
        else
            trial_i = trial_i + 1
        end if
    end for
    Onlooker Bees
    Compute selection probability: p_i = f_i / Σ f_i
    for each onlooker do
        Select solution using roulette wheel
        Generate neighbor and apply greedy update
    end for
    Scout Bees
    for i = 1 to SN do
        if trial_i ≥ L then
            Replace θ_i with random solution
            Evaluate fitness
            trial_i = 0
        end if
    end for
end for
θ* = arg max f_i
Retrain final hybrid model using θ*
return θ*
```

4. Experimental Setup

The datasets, computational environment, evaluation methodology, and baseline models used to assess the performance of the proposed ABC-optimized hybrid architecture in this section. All experiments were conducted using real-world code-mixed social media data without any synthetic augmentation to ensure practical relevance and reproducibility. All experiments were conducted using fixed random seeds to ensure reproducibility.

4.1. Datasets

The publicly available Hindi-English (Hinglish) hate speech corpora from social media platforms were employed to test the proposed framework. The corpora demonstrate

considerable linguistic diversity, such as the use of informal language, phonetic transliteration, spelling differences, abbreviations, and prevalent code-switching between Hindi and English. The Code-Mixed Hinglish Hate Speech Detection Corpus, which contains approximately 29,550 social media postings annotated with binary labels signifying hate/offensive or non-hate speech, is the principal corpus employed in this research. This dataset is publicly available through Kaggle Dhekane (2026).

To further evaluate the generalization capability of the proposed model, additional benchmark datasets from the HASOC (Hate Speech and Offensive Content) shared tasks conducted under the FIRE evaluation campaign (2019–2021) were also considered. These datasets include multilingual and code-mixed Hindi–English social media posts annotated for hate and offensive language HASOC FIRE 2024 (2024).

The datasets were preprocessed to enhance the quality of the data and make them transformer-friendly before the training of the model. These included whitespace normalization, case conversion to lowercase, removal of URLs, mentions, hashtags, emoticons, and special characters. The preprocessed text was tokenized using the DeBERTa tokenizer from the HuggingFace library. The datasets were split into training, validation, and testing sets with a 70:15:15 split.

4.2. Hardware and Software Environment

All experiments were conducted on a standard computing platform to ensure the practical feasibility and reproducibility of the proposed approach in moderate-resource environments. The system configuration used for model development and evaluation is summarized as follows:

- Processor: Intel Core i7 CPU
- RAM: 16 GB
- GPU: NVIDIA GPU with 6 GB memory
- Operating System: Windows/Linux
- Programming Language: Python 3.9
- Deep Learning Framework: PyTorch
- Transformer Library: HuggingFace Transformers
- Supporting Libraries: NumPy, Pandas, and Scikit-learn

The proposed hybrid DeBERTa–RNN–LSTM model was trained using GPU acceleration. To improve computational efficiency and reduce memory consumption, mixed precision training (FP16) was employed using automatic mixed precision. Gradient accumulation was used to simulate larger effective batch sizes while operating within the available GPU memory limits. In addition, pinned memory was enabled in the data loader to accelerate CPU-to-GPU data transfer.

To further reduce computational overhead and stabilize training, only the top six layers of the pretrained DeBERTa encoder were fine-tuned, while the remaining lower layers were frozen. This partial fine-tuning strategy reduces the number of trainable parameters while allowing effective adaptation to the target task. The maximum input sequence length was also limited to control the computational cost associated with the self-attention mechanism.

Model training was performed using the Adam optimizer with cross-entropy loss, where the learning rate was determined through the Artificial Bee Colony optimization process. During optimization, each candidate hyperparameter configuration was trained for a fixed number of epochs, and the macro F1-score obtained on the validation set was used as the fitness value. Fixed random seeds were used to ensure experimental reproducibility.

Hyperparameter optimization was performed using ABC algorithm. The validation is used to evaluate the fitness of each food source using macro F1-score, which is a hyperparameter vector candidate. The population-based search process of ABC uses probabilistic selection and local search to effectively explore the hyperparameter space. To prevent premature convergence and maintain diversity, the search process is split into three phases: the employed bee phase, which is used for local search; the onlooker bee phase, which is used to optimize promising solutions; and the scout bee phase, which is used to generate new solutions. The optimization was possible for resource-constrained scenarios because it employed a population size of 10 with 20 cycles and an abandonment limit of 5.

5. Results and Discussion

The experimental results of the ABC-optimized hybrid DeBERTa-RNN-LSTM model on Hinglish hate speech classification are presented in this section. The performance of the model has been tested using different classification metrics on the test data. For a comprehensive analysis of the model, it also presents training results, error analysis, and confidence analysis.

5.1. Dataset Characteristics

The exploratory analysis of the dataset is shown in Figure 3. The class distribution indicates a moderately balanced dataset, with a slightly higher proportion of non-hate samples compared to hate samples. The dataset contains multilingual and code-mixed content, with English and Hindi being the dominant languages.

The text length distribution shows that most samples are short social media posts, typically within 50–150 characters, reflecting real-world user-generated content. Such variability in language usage and text length makes the hate-speech detection task more challenging.

5.2. Overall Performance

The best hyperparameter configuration identified by the Artificial Bee Colony optimization achieved the following performance on the test set:

- Test Accuracy: 71.73%

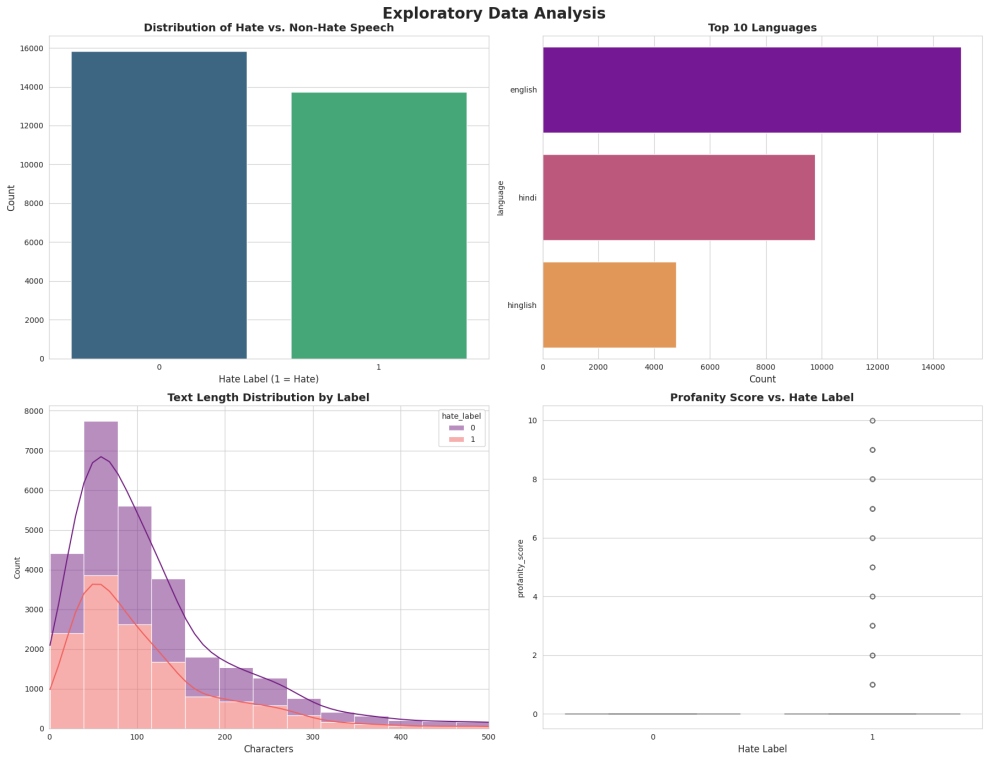

Figure 3: Exploratory data analysis of the Hinglish hate-speech dataset showing (i) class distribution of hate and non-hate samples, (ii) dominant language composition, (iii) text length distribution across classes, and (iv) profanity score variation with respect to hate labels.

Table 2
Depend on the test set, Classification performance

Class	Precision	Recall	F1-score	Support
Non-Hate	0.7110	0.7957	0.7509	2374
Hate	0.7269	0.6270	0.6733	2059
Accuracy		0.7173		
Macro Avg	0.7189	0.7114	0.7121	4433
Weighted Avg	0.7184	0.7173	0.7149	4433

- Macro F1-score: 0.6733
- Test AUC: 0.7951
- Validation AUC (best): 0.8016

These results show the model's effectiveness in understanding the language and semantic characteristics of the code-mixed social media messages. The model handles the informal structure and code-mixing present in the Hinglish datasets. As per the classification experiment (shown in Table 2), it is able to classify hate speech and non-hate speech with minimal bias.

The result shows that the proposed model has little higher precision and lower recall value for the hate class. This indicates that there could be some difficulties in identifying the hate speech.

5.3. Confusion Matrix Analysis

A complete view of class-wise predictions is shown in "Figure 4". The model correctly identified 62.7% (1291 cases) of hate speech and 79.6% (1889 cases) of non-hate speech. However, 768 hate speech samples are misclassified as non-hatred, indicating that the model is still having problems with implicit or context-dependent hate emotions.

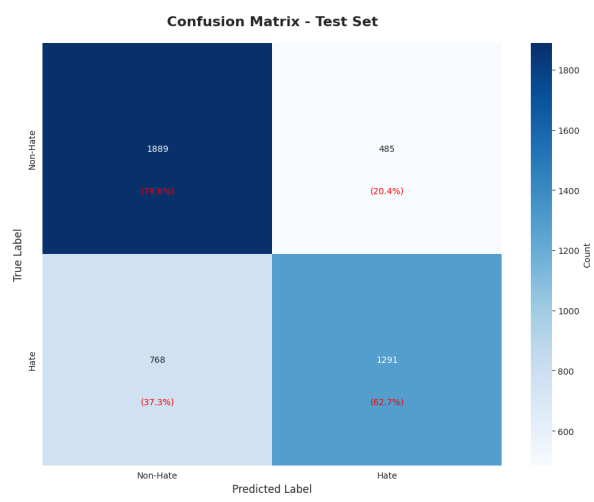

Figure 4: Confusion matrix illustrating the proposed model's performance on the test set.

The model correctly classified:

- 1889 non-hate instances (79.6%)
- 1291 hate instances (62.7%)

In cases where there is strong emotional or critical language but little hate speech, there are also false positives

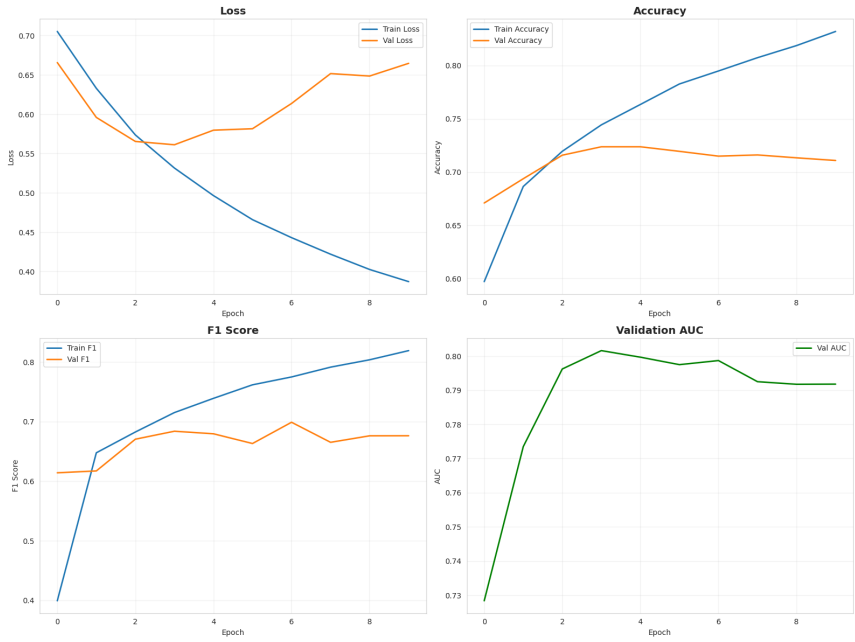

Figure 5: The proposed model's Training and validation performance across epochs showing (i) loss convergence, (ii) accuracy progression, (iii) F1-score improvement, and (iv) validation AUC behavior.

(485 cases). Taking everything into account, the confusion matrix indicates that the model is better at identifying non-hate speech while still having a decent chance of identifying hate speech. However, 768 hate speech samples are misclassified as non-hatred, indicating that context-dependent or implicit hate speech is still difficult to identify.

5.4. Training Behavior

The training process dynamics, shown in Figure 5, illustrate a steady decrease in the training loss function, with the validation accuracy converging after the first training epochs, reaching a high value of validation AUC of about 0.80. The mild overfitting is revealed by the negligible difference between the training and validation curves, which is mitigated by the application of dropout, partial transformer fine-tuning, and hyperparameter optimization using the ABC method.

The maximum value of the validation AUC of 0.8016 was achieved at the middle training phase, and the F1-score dynamics illustrate steady learning and progressive improvement in the classification accuracy.

5.5. Qualitative Prediction Examples

Illustrative prediction examples in Figure 6 demonstrate how the proposed model performs under varying linguistic and semantic conditions, including hate speech and non-hate speech, to analyze its performance. Although obviously derogatory language is assigned a higher hatred probability, the model performs well in understanding the context by producing highly confident predictions for obvious attitudes. Nevertheless, situations involving ambiguity or contextuality, such as Hinglish phrases, result in medium confidence values, highlighting the limitations in hate speech detection on social media platforms. This qualitative analysis proves the model's capability to process multilingual and code-mixed text with confidence-aware predictions, supporting the quantitative analysis.

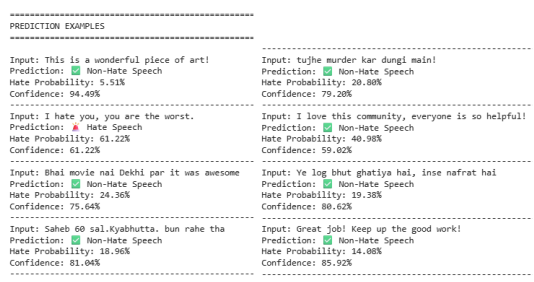

Figure 6: Sample prediction outputs of the proposed model.

5.6. Confidence-Based Performance Analysis

The performance of the model was then analyzed for different intervals of prediction confidence. It was found that predictions made with higher levels of confidence have better accuracy:

- Confidence 0.5–0.6: 51.08% accuracy
- Confidence 0.6–0.7: 66.37% accuracy
- Confidence 0.7–0.8: 66.67% accuracy
- Confidence 0.8–0.9: 83.54% accuracy
- Confidence 0.9–1.0: 94.87% accuracy

This indicates that predictions made with high confidence levels are very reliable and the model is well-calibrated.

5.7. Error Analysis

1,253 out of 4,433 samples (28.27%) were mistakenly classified by the model, of which 485 were false positives and 768 were false negatives. The primary reasons for false negatives are implicit or sarcastic hate speech, situation-dependent obscene language, and code-mixed words with reduced formal spelling. If critical or emotional language is employed without explicit hate speech, it may lead to false positives. The model is strong at explicit hate speech and neutral texts, but it struggles with those that are ambiguous or in between, as stated in the analysis.

To gain insights into the limitations of the proposed model, qualitative error analysis was performed. Figures 7 and 8 depict examples of false negative and false positive predictions, respectively. False negatives refer to hate situations that are predicted to be non-hate. Most of these cases involve context abuse, sarcasm, implied hate, or complex code-mixed messages which is shown in Figure 7. Since the malicious intent is not explicitly expressed, such cases are difficult to detect. A false positive (shown in Figure 8) occurs when non-hateful speech is mistakenly identified as hate speech, especially in cases involving strong emotional speech or ambiguity. The model struggles with casual online social communication, particularly in terms of awareness of context and the identification of implicit hate speech. When hate speech is labeled as non-hate speech, particularly due to factors such as code-mixing and sarcasm, it is referred to as a false negative. It is vital to improve these areas to increase the accuracy of the model to detect hate speech.

```
==================================================
EXAMPLE ERRORS (False Negatives)
==================================================

Text: चीनी कंपनी लुप्मो ने एक केक जारी किया है जिसके अंदर एक टैबलेट छिपाया है जिससे बच्चों को लकवा हो जाता
True Label: Hate | Predicted: Non-Hate | Probability: 43.31%

Text: पक्षों में मारपीट बच्चों के विवाद में हुई दोनों पक्षों में एकएक लोग घायल दोनों पक्षों ने कोतवाली में
True Label: Hate | Predicted: Non-Hate | Probability: 37.98%

Text: आज हमसे पूछा जा रहा है की आपके कुत्तुर्गों का साबुत क्या है अरे दिल्ली में इतनी बड़ी सात बारा है ना मुस
True Label: Hate | Predicted: Non-Hate | Probability: 25.88%

Text: no this is a group effort wikipedia is a collaborative community and there are no school essays here
True Label: Hate | Predicted: Non-Hate | Probability: 36.11%

Text: why year old angry what s he brother does those boys go to i don t know this an asshole
True Label: Hate | Predicted: Non-Hate | Probability: 37.17%
```

Figure 7: Examples of false negative predictions where hate speech instances are incorrectly classified as non-hate. Many errors involve implicit or context-dependent hate expressions.

```
==================================================
EXAMPLE ERRORS (False Positives)
==================================================

Text: केवल निचली जात के लोग ही इतने अनपढ़ हो सकते हैं
True Label: Non-Hate | Predicted: Hate | Probability: 74.74%

Text: if it walks like a duck and quacks like a duck it s either guest speaking or in fact a duck
True Label: Non-Hate | Predicted: Hate | Probability: 71.58%

Text: no decatur tx thanks dawg you spelled awkward wrong hot dog condom style the only ex laughing big gr
True Label: Non-Hate | Predicted: Hate | Probability: 85.22%

Text: वास्तव मेंहीजड़े ही सबसे टट्टी होते हैं
True Label: Non-Hate | Predicted: Hate | Probability: 73.61%

Text: i asked harshita ap dekhte ho she said kabhi kabhi me apne wo scn dekha jisme tmhari rago m kon sa k
True Label: Non-Hate | Predicted: Hate | Probability: 68.13%
```

Figure 8: Examples of false positive predictions.

5.8. Effectiveness of ABC Optimization

The Artificial Bee Colony optimization method enhanced the proposed framework by successfully determining the best hyperparameters, achieving a validation AUC of 0.8016, and ensuring the right generalization performance. The method improved the stability of models, accuracy, and training performance while maintaining low computing expenses by appropriately exploring the hyperparameter space and achieving a balance between exploration and exploitation. A sound system for code-mixed hate speech detection was developed by integrating contextual transformer features with ABC optimization, balancing recall and precision, and satisfying practical computing constraints. However, some problems still exist in the detection of implicit and context-dependent hate speech, indicating that incorporating conversational context, external knowledge, or contrastive learning could further enhance detection performance.

6. Conclusion and Future Work

An Artificial Bee Colony optimized hybrid deep learning framework was proposed with the aim of hate speech detection in code-mixed Hindi-English social media content in this paper. The proposed framework incorporates the ABC optimization method for hyperparameter tuning and integrates RNN and LSTM layers for modeling sequential data with DeBERTa for contextual understanding. The test accuracy achieved was 71.73%, test AUC was 0.7951, and macro F1-score was 0.6733, as per the assessment done on Hinglish datasets. As far as the identification of non-hate speech is concerned, the proposed framework performs better (79.6%) than when it is concerned with hate speech (62.7%). The identification of implicit hate speech and context-dependent offensiveness is a challenge. Future work will involve exploring the use of lightweight transformer variants, contrastive learning, and the addition of conversational context. The proposed framework can be used in automatic content moderation systems and serves as a viable solution for hate speech identification in multilingual scenarios.

References

Badjatiya, P., Gupta, S., Gupta, M., Varma, V., 2017. Deep learning for hate speech detection in tweets, in: Proceedings of the 26th International Conference on World Wide Web Companion.

Bohra, A., Vijay, D., Singh, V., Akhtar, S.S., Shrivastava, M., 2018. A dataset of hindi-english code-mixed social media text for hate speech detection, in: Proceedings of the Workshop on Computational Modeling of People's Opinions, Personality, and Emotions in Social Media, pp. 36–41.

Caselli, T., Basile, V., Mitrović, J., Granitzer, M., 2020. Hatebert: Retraining bert for abusive language detection in english, in: Proceedings of the Fifth Workshop on Online Abuse and Harms (WOAH).

Chopra, S., Sawhney, R., Mathur, P., Shah, R.R., 2020. Hindi-english hate speech detection: Author profiling, debiasing, and practical perspectives, in: Proceedings of the AAAI conference on artificial intelligence, pp. 386–393.

Davidson, T., Warmsley, D., Macy, M., Weber, I., 2017. Automated hate speech detection and the problem of offensive language, in: Proceedings of the International AAAI Conference on Web and Social Media (ICWSM).

Dedeturk, B.K., Akay, B., 2025. A parallel hybrid approach integrating clonal selection with artificial bee colony for logistic regression in spam email detection. Neural Computing and Applications 37, 22401–22419.

Devlin, J., Chang, M.W., Lee, K., Toutanova, K., 2019. Bert: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of NAACL-HLT.

Dhekane, S., 2026. Code-mixed hinglish hate speech detection dataset. Kaggle. URL: https://www.kaggle.com/datasets/sharduldhekane. accessed: Feb 18, 2026.

Fortuna, S., Nunes, S., 2018. A survey on automatic detection of hate speech in text. ACM Computing Surveys 51.

Hakli, H., Kiran, M.S., 2020. An improved artificial bee colony algorithm for balancing local and global search behaviors in continuous optimization. International Journal of Machine Learning and Cybernetics 11, 2051–2076.

HASOC FIRE 2024, 2024. Hate speech and offensive content identification: Hasoc 2024 dataset. Forum for Information Retrieval Evaluation (FIRE). URL: https://hasocfire.github.io/hasoc/2024/dataset.html. accessed: January 2026.

Jagdale, S., Inamdar, M., Takalikar, G., Khade, O., Joshi, R., 2024. On importance of code-mixed embeddings for hate speech identification, in: International Conference on Computing and Communication Networks, Springer. pp. 269–277.

Kamble, S., Joshi, A., 2018. Hate speech detection from code-mixed hindi-english tweets using deep learning models, in: Proceedings of the 15th International Conference on Natural Language Processing (ICON), NLP Association of India, International Institute of Information Technology, Hyderabad, India. pp. 150–155. URL: https://aclanthology.org/2018.icon-1.22/.

Karaboga, D., Akay, B., 2011. A modified artificial bee colony (abc) algorithm for constrained optimization problems. Applied Soft Computing 11, 3021–3031.

Karaman, A., Karaboga, D., Pacal, I., Akay, B., Basturk, A., Nalbantoglu, U., Coskun, S., Sahin, O., 2023a. Hyper-parameter optimization of deep learning architectures using artificial bee colony (abc) algorithm for high performance real-time automatic colorectal cancer polyp detection. Applied Intelligence 53, 15603–15620.

Karaman, A., Pacal, I., Basturk, A., Akay, B., Nalbantoglu, U., Coskun, S., Sahin, O., Karaboga, D., 2023b. Robust real-time polyp detection system design based on yolo algorithms by optimizing activation functions and hyper-parameters with artificial bee colony (abc). Expert Systems with Applications 221, 119741.

Kaya, E., Gorkemli, B., Akay, B., Karaboga, D., 2022. A review on the studies employing artificial bee colony algorithm to solve combinatorial optimization problems. Engineering Applications of Artificial Intelligence 115, 105311.

Kiran, M.S., 2021. A binary artificial bee colony algorithm and its performance assessment. Expert Systems with Applications 175, 114817.

Kiran, M.S., Findik, O., 2015. A directed artificial bee colony algorithm. Applied Soft Computing 26, 454–462.

Kiran, M.S., Hakli, H., Gunduz, M., Uguz, H., 2015. Artificial bee colony algorithm with variable search strategy for continuous optimization. Information Sciences 300, 140–157.

Lin, Y., Wang, J., Li, X., Zhang, Y., Huang, S., 2021. An improved artificial bee colony for feature selection in qsar. Algorithms 14, 120.

Patel, V., Tiwari, A., Patel, A., 2016. A comprehensive survey on hybridization of artificial bee colony with particle swarm optimization algorithm and abc applications to data clustering, in: Proceedings of the International Conference on Informatics and Analytics, pp. 1–9.

Patwa, P., et al., 2021. Overview of the hasoc subtrack at fire 2021: Hate speech and offensive content identification in english and indo-aryan languages, in: Forum for Information Retrieval Evaluation (FIRE).

Santosh, T., Aravind, K., 2019. Hate speech detection in hindi-english code-mixed social media text, in: Proceedings of the ACM India joint international conference on data science and management of data, pp. 310–313.

Schmidt, A., Wiegand, M., 2017. A survey on hate speech detection using natural language processing, in: Proceedings of the Fifth International Workshop on Natural Language Processing for Social Media.

Sharma, A., Bhalla, R., 2025. Detecting hate speech for hindi-english code-mix text data using dual contrastive learning. Procedia Computer Science 259, 35–43.

Taghizadeh-Mehrjardi, R., Neupane, R., Sood, K., Kumar, S., 2017. Artificial bee colony feature selection algorithm combined with machine learning algorithms. Carbon Management 8, 277–291.

Uzer, M.S., Yilmaz, N., Inan, O., 2013. Feature selection method based on artificial bee colony algorithm and support vector machines for medical datasets classification. The Scientific World Journal , 419187.

Vidgen, B., Yasseri, T., Margetts, H., 2019. Challenges and frontiers in abusive content detection, in: Proceedings of the Third Workshop on Abusive Language Online (ALW3).

Wang, X.h., Zhang, Y., Sun, X.y., Wang, Y.l., Du, C.h., 2020. Multi-objective feature selection based on artificial bee colony: An acceleration approach with variable sample size. Applied Soft Computing 88, 106041.

Waseem, Z., Hovy, D., 2016. Hateful symbols or hateful people? predictive features for hate speech detection on twitter, in: Proceedings of NAACL-HLT.

Young, T., Hazarika, D., Poria, S., Cambria, E., 2018. Recent trends in deep learning based natural language processing. IEEE Computational Intelligence Magazine 13.

Zhang, Z., Fu, Y., Gao, K., Pan, Q., Huang, M., 2024. A learning-driven multi-objective cooperative artificial bee colony algorithm for distributed flexible job shop scheduling problems with preventive maintenance and transportation operations. Computers & Industrial Engineering 196, 110484.

Zhang, Z., Robinson, D., Tepper, J., 2018. Detecting hate speech on twitter using a convolution-gru based deep neural network, in: European Semantic Web Conference (ESWC).

Declaration of interests

☒The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

☐The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: